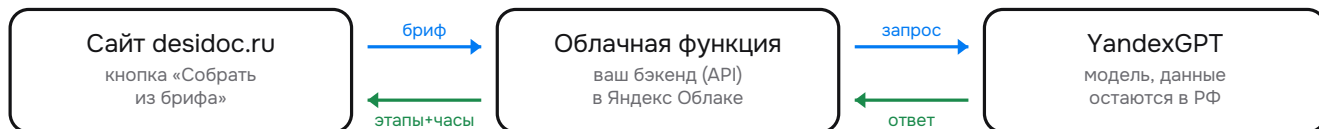


AI-разбор брифа для DesiDoc

Пошаговая инструкция: подключаем YandexGPT через Яндекс Облако — свой бэкенд (API), без хранения ключей и без передачи данных за рубеж.



Наверх уходит бриф (без перс. данных), обратно — JSON с этапами и часами.
Сумму считает сам калькулятор: часы x ставка + налог. Модель денег не считает.

Что понадобится: аккаунт в Яндекс Облаке (уже есть), около 40 минут. Стоимость: примерно 0,2–0,8 ₽ за один разбор брифа; тысяча смет — несколько сотен рублей.

Нюанс. Картинки в инструкции — схемы экранов, а не скриншоты. Консоль Яндекса иногда меняет названия кнопок; ориентируйтесь по смыслу. Красные кружки (1)(2)(3) — места, куда смотреть и жать.

Маршрут: Часть 1 — доступ к модели (шаги 1–5) · Часть 2 — бэкенд/API (шаги 6–10) · Часть 3 — сайт (шаги 11–14) · Часть 4 — политика (шаги 15–16) · Часть 5 — проверка (шаг 17). В конце: полный код, решение проблем, чек-лист.

Часть 1. Доступ к YandexGPT

Шаг 1. Вход в консоль

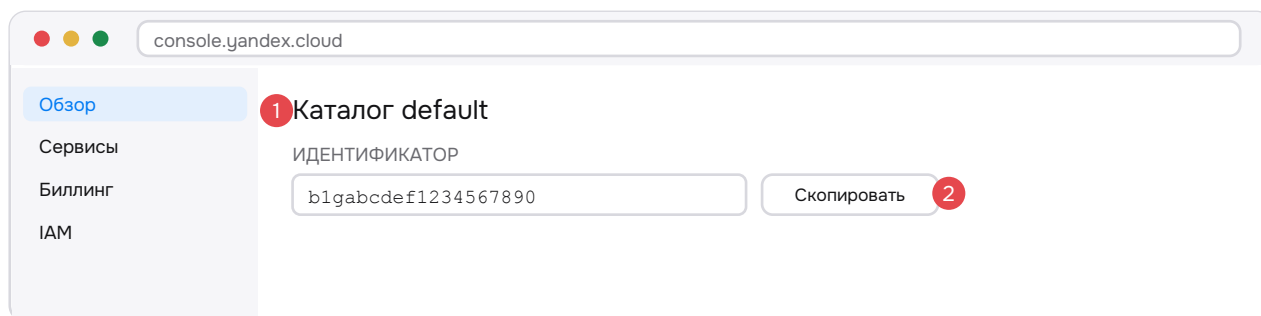
Откройте `console.yandex.cloud` и войдите под своим Яндекс-аккаунтом. Аккаунт у вас уже одобрен – просто входим.

Шаг 2. Проверить платёжный аккаунт

Слева в меню найдите Billing (Платежи). Статус должен быть ACTIVE или «Пробный период». Если так – идём дальше.

Шаг 3. Узнать ID каталога (FOLDER_ID)

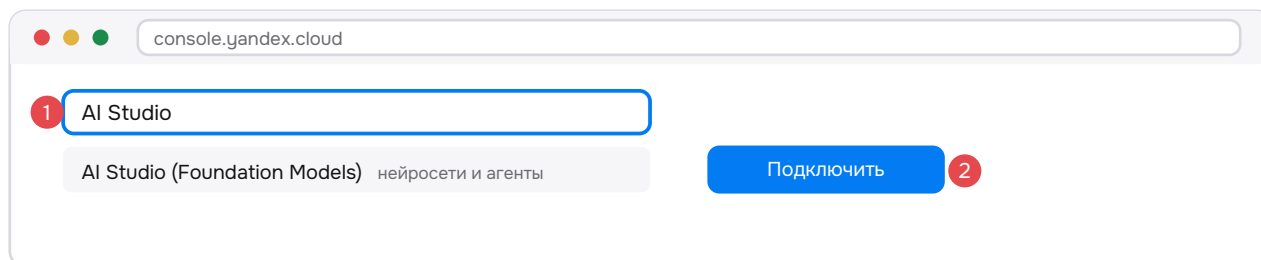
На главной странице консоли выберите своё облако – внутри будет каталог (обычно называется default). Нужен именно каталог, не облако! Откройте его.



(1) Название каталога – вы внутри каталога default.

(2) Строка «Идентификатор» – нажмите «Скопировать». Это ваш FOLDER_ID (вид: b1g...). Запишите его – понадобится на шаге 8.

Шаг 4. Включить YandexGPT (AI Studio)

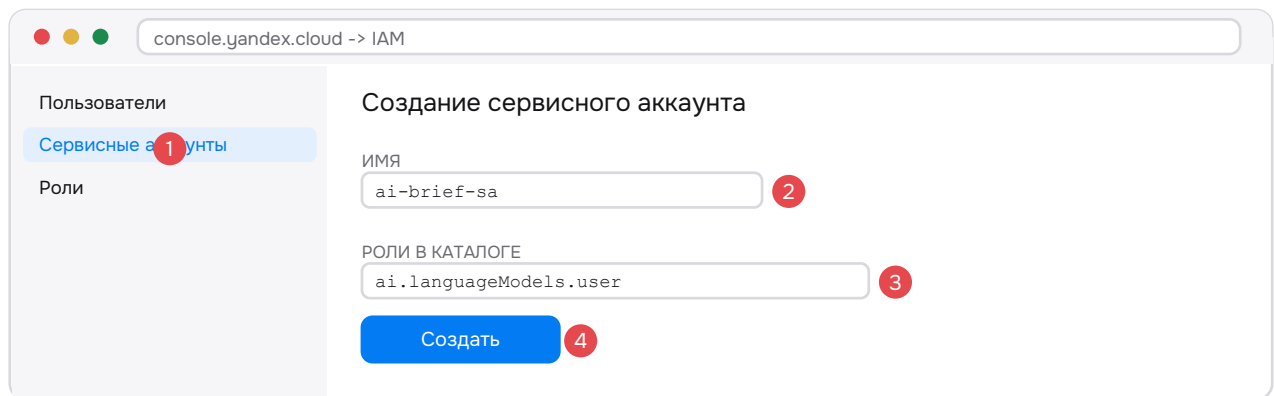


(1) В поиске сверху наберите AI Studio (старое название – Foundation Models) и откройте сервис.

(2) Если появится кнопка «Подключить» или экран с условиями – нажмите и примите условия. После этого модель доступна в вашем каталоге.

Шаг 5. Сервисный аккаунт (доступ функции к модели)

Это «технический пользователь», от имени которого функция будет ходить к модели. Благодаря ему не нужно создавать и прятать API-ключи.



- (1) В поиске наберите IAM > раздел «Сервисные аккаунты» > «Создать сервисный аккаунт».
- (2) Имя: `ai-brief-sa` (можно любое, но так будет совпадать с инструкцией).
- (3) Нажмите «Добавить роль» и выберите роль `ai.languageModels.user` — начните печатать «`ai.lang...`», она найдётся.
- (4) Нажмите «Создать». Готово — ключей создавать не нужно.

Нюанс. Роль пишется именно так: `ai.languageModels.user`. Если взять похожую (например, `viewer`) — функция получит ошибку доступа.

Часть 2. Бэкенд (API): облачная функция

Функция — это маленький сервер, который живёт в Яндекс Облаке и просыпается только на время запроса. Платить за «простой» не нужно. Она принимает бриф с сайта, вырезает персональные данные, спрашивает YandexGPT и возвращает JSON с этапами.

Шаг 6. Создать функцию

- (1) В поиске наберите Cloud Functions > «Создать функцию».
- (2) Имя: ai-brief-proxy > «Создать».

Шаг 7. Вставить код

Откройте вкладку Редактор. Выберите среду выполнения Node.js 20. В файл index.js вставьте ПОЛНОСТЬЮ код из Приложения А (он же лежит в проекте: experiments/ai-brief/cloud-function/index.js).

Шаг 8. Настроить версию

- (1) Среда выполнения: Node.js 20.
- (2) Точка входа: index.handler — именно так, с точкой посередине.
- (3) Таймаут: 30 секунд. Сервисный аккаунт: выберите ai-brief-sa из списка — это самый важный пункт!
- (4) Переменные окружения: добавьте FOLDER_ID = ваш ID из шага 3 и ALLOWED_ORIGIN = https://desidoc.ru.
- (5) Нажмите «Создать версию» и подождите несколько секунд.

Юанс. Необязательная переменная YAGPT_MODEL: значение yandexgpt — модель Pro (умнее, стоит дороже), yandexgpt-lite — Lite (дешевле). Если переменную не добавлять, используется Pro.

Шаг 9. Сделать функцию публичной и взять ссылку

функция ai-brief-proxy -> Обзор

Публичная функция ☒ 1

ССЫЛКА ДЛЯ ВЫЗОВА

`https://functions.yandexcloud.net/d4e...` Скопировать 2

- (1) Вкладка «Обзор» > включите тумблер «Публичная функция». Без этого сайт получит ошибку 403.
- (2) Скопируйте ссылку для вызова – вид `https://functions.yandexcloud.net/d4e...` Это адрес вашего API. Запишите – понадобится на шаге 12.

Шаг 10. Проверить функцию (ещё без сайта)

функция ai-brief-proxy -> Тестирование

ТЕЛО ЗАПРОСА

`{ "brief": "Лендинг для кофейни, 5 экранов,
нужен дизайн и графики" }` 1

Запустить тест 2

`{ "estimate_title": "...", "stages": [
 { "name": "Брифинг", "hours": 3 }, ...] }` 3

- (1) Вкладка «Тестирование»: в поле тела запроса вставьте JSON с брифом (как на схеме).
- (2) Нажмите «Запустить тест».
- (3) Успех – в ответе JSON с полем `stages` (этапы и часы). Если вместо этого ошибка – загляните в Приложение В «Решение проблем».

Нюанс. Если во вкладке Тестирование просят «тип данных» – выберите HTTPS / http-запрос, метод POST, и вставьте JSON в тело.

Часть 3. Подключить сайт

Правим один файл: `js/designkit.js`. Совет: сначала потренируйтесь на копии в `experiments/unit-pricing/js/designkit.js`.

Шаг 11. Переименовать старую логику в запасную

Найдите в файле функцию `async function analyzeBrief(text)` и переименуйте её в `analyzeBriefLocal`. Она останется запасным вариантом: если интернет или функция недоступны – калькулятор всё равно посчитает.

Шаг 12. Добавить новую функцию `analyzeBrief`

Вставьте над `analyzeBriefLocal` код ниже и подставьте свою ссылку из шага 9 в первую строку:

```
const AI_BRIEF_ENDPOINT = "https://functions.yandexcloud.net/dXXXXXXXXX"; // <-- ваша ссылка (шаг 9)

async function analyzeBrief(text) {
  const source = String(text || "").trim();
  if (!source) throw new Error("Добавьте бриф или сообщение клиента.");
  try {
    const res = await fetch(AI_BRIEF_ENDPOINT, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ brief: source }),
    });
    if (!res.ok) throw new Error("bad status");
    const data = await res.json();
    if (!data || !Array.isArray(data.stages) || !data.stages.length) throw new Error("empty");
    return data; // формат тот же – applyBriefAnalysis не меняем
  } catch (e) {
    return analyzeBriefLocal(source); // запасной вариант, если AI не ответил
  }
}
```

Шаг 13. Включить кнопку AI

Найдите строку `const BRIEF_AI_ENTRY_ENABLED = false;` и поменяйте `false` на `true`.

Шаг 14. Поднять версию файла

В `index.html` найдите строку с `designkit.js?v=...` и увеличьте число на 1. Иначе браузеры покажут старую версию из кэша, и кнопка не появится.

Часть 4. Согласие и политика (обязательно)

Шаг 15. Предупреждение в окне брифа

В окне ввода брифа покажите текст (удобно через существующую строку статуса `setBriefAiStatus`):

Важно! «Текст обрабатывается ИИ для оценки сметы. Не вставляйте персональные данные третьих лиц (телефоны, почту, ФИО клиентов).»

Шаг 16. Строка в политику

В `policy.html` добавьте пункт:

Важно! «Для предварительной оценки сметы введённый текст обрабатывается языковой моделью YandexGPT (Yandex Cloud, РФ). Текст не сохраняется. Не вводите персональные данные третьих лиц.»

Финальные формулировки политики стоит показать юристу — разовая недорогая страховка на фоне нынешних штрафов по 152-ФЗ.

Часть 5. Финальная проверка

Шаг 17. Проверяем всё вместе

1. Откройте сайт (убедитесь, что загрузилась новая версия — шаг 14).
2. Выберите проект «Другое» — появится кнопка «Собрать из брифа».
3. Вставьте: «Презентация на 15 слайдов для стартапа, есть графики и схемы».
4. Нажмите «Рассчитать».

Проверка. Должны появиться этапы с часами, и сумма посчитается автоматически. Затем выключите интернет и повторите — сработает запасной вариант без ошибок.

Приложение А. Полный код облачной функции

Скопируйте всё содержимое в файл `index.js` в редакторе функции (шаг 7). Этот же файл лежит в проекте: `experiments/ai-brief/cloud-function/index.js`.

```
// Облачная функция Яндекса — «переходник» между сайтом и YandexGPT.
// Держит доступ к модели, вырезает персональные данные, ничего не хранит.
// Runtime: Node.js 20. Точка входа: index.handler
// Переменная окружения: FOLDER_ID (ID каталога в Яндекс Облаке)
// Сервисный аккаунт с ролью ai.languageModels.user должен быть привязан к функции.

const FOLDER_ID = process.env.FOLDER_ID;
// "yandexgpt" = Pro (умнее), "yandexgpt-lite" = Lite (дешевле). Можно поменять переменной
  YAGPT_MODEL.
const MODEL = process.env.YAGPT_MODEL || "yandexgpt";

// Что разрешаем сайту обращаться к функции. Лучше указать свой домен вместо "*".
const ALLOWED_ORIGIN = process.env.ALLOWED_ORIGIN || "https://desidoc.ru";

const CORS = {
  "Access-Control-Allow-Origin": ALLOWED_ORIGIN,
  "Access-Control-Allow-Methods": "POST, OPTIONS",
  "Access-Control-Allow-Headers": "Content-Type",
};

const SYSTEM_PROMPT = [
  "Ты — ассистент дизайнера-фрилансера. По брифу клиента собери структуру сметы.",
  "Верни ТОЛЬКО валидный JSON без пояснений, по схеме:",
  '{',
  '  "estimate_title": "короткое название сметы",' ,
  '  "project_type":',
  '    "website|logo|brand_identity|packaging|presentation|smm|marketplaces|outdoor|print|custom"',
  '  ,',
  '  "brand_name": "имя бренда или пустая строка",' ,
  '  "stages": [{ "name": "этап", "description": "что делается", "hours": число }],',
  '  "pricing_risks": ["возможный риск по цене"],',
  '  "additional_costs": ["возможный доп. расход"]',
  '}',
  "Оцени часы реалистично для ОДНОГО дизайнера. Сделай 5-8 этапов.",
  "Деньги и итоговую сумму НЕ считай — только часы.",
].join("\n");

// Убираем персональные данные из текста ДО отправки в модель (второй рубеж защиты).
function stripPII(text) {
  return String(text)
    .replace(/[\\w.+]+@[\\w-]+\\.([\\w.-]+)/g, "[email]")
    .replace(/[?:\\+7|8][\\s\\-()]*\\d{3}[\\s\\-()]*\\d{3}[\\s\\-()]*\\d{2}[\\s\\-()]*\\d{2}/g, "[телефон]")
    .replace(/@[a-zA-Z0-9_]{3,}/g, "[ник]");
}

module.exports.handler = async (event, context) => {
  const method = event.httpMethod || "POST";
  if (method === "OPTIONS") return { statusCode: 204, headers: CORS, body: "" };
  if (method !== "POST") return { statusCode: 405, headers: CORS, body: "Method Not Allowed" };

  // Тело запроса (может прийти в base64 — обрабатываем оба случая).
  let raw = event.body || "{}";
  if (event.isBase64Encoded) raw = Buffer.from(raw, "base64").toString("utf-8");

  let brief = "";
  try { brief = String(JSON.parse(raw).brief || "").trim(); } catch (e) {}

  if (!brief || brief.length > 4000) {
    return json(400, { error: "Некорректный бриф" });
  }

  // IAM-токен привязанного сервисного аккаунта — ключ хранить НЕ нужно.
```



```

const token = context.token && context.token.access_token;
if (!token) return json(500, { error: "Нет токена сервисного аккаунта" });

let resp;
try {
  resp = await fetch("https://llm.api.cloud.yandex.net/foundationModels/v1/completion", {
    method: "POST",
    headers: {
      "Authorization": `Bearer ${token}`,
      "Content-Type": "application/json",
      "x-folder-id": FOLDER_ID,
    },
    body: JSON.stringify({
      modelUri: `gpt://${FOLDER_ID}/${MODEL}/latest`,
      completionOptions: { stream: false, temperature: 0.2, maxTokens: "2000" },
      messages: [
        { role: "system", text: SYSTEM_PROMPT },
        { role: "user", text: stripPII(brief) },
      ],
    }),
  });
} catch (e) {
  return json(502, { error: "AI недоступен" });
}

if (!resp.ok) return json(502, { error: "AI недоступен" });

const data = await resp.json();
const text = data?.result?.alternatives?.[0]?.message?.text || "{}";

// Достаём JSON из ответа модели (на случай, если вокруг него будет лишний текст).
const match = text.match(/\{([\s\S]*\})/);
const payload = match ? match[0] : "{}";

return {
  statusCode: 200,
  headers: { ...CORS, "Content-Type": "application/json; charset=utf-8" },
  body: payload,
};
};

function json(statusCode, obj) {
  return {
    statusCode,
    headers: { ...CORS, "Content-Type": "application/json; charset=utf-8" },
    body: JSON.stringify(obj),
  };
}

```

Приложение В. Решение проблем

Что видите	Причина	Что делать
«Нет токена сервисного аккаунта»	К версии функции не привязан сервисный аккаунт	Шаг 8, пункт (3): выберите ai-brief-sa и создайте версию заново
«AI недоступен» (ошибка 502)	Неверный FOLDER_ID, модель не активирована или нет роли	Проверьте шаги 3–5 и переменную FOLDER_ID в шаге 8
«Некорректный бриф» (400)	Пустой текст или длиннее 4000 символов	Сократите бриф
Ошибка 403 при вызове по ссылке	Функция не публичная	Шаг 9: включите тумблер
В консоли браузера ошибка CORS	ALLOWED_ORIGIN не совпадает с адресом сайта	Поставьте https://desidoc.ru (на время локальных тестов можно *)
Кнопки «Собрать из брифа» нет	Флаг выключен или браузер держит старый файл	Шаги 13–14: включите флаг и поднимите версию
Этапы пришли, но часы странные	Модель оценивает «средне по рынку»	Это нормально: часы – прикидка, поправьте под свою скорость

Приложение С. Чек-лист безопасности (152-ФЗ)

- [x] Данные остаются в РФ (YandexGPT) – транграничной передачи нет.
- [x] Персональные данные вырезаются в функции до отправки в модель (stripPII).
- [x] Функция ничего не сохраняет – нет базы персональных данных.
- [x] Ключи не хранятся – доступ через сервисный аккаунт.
- [] Предупреждение в окне брифа добавлено (шаг 15).
- [] Строка в политике добавлена (шаг 16).
- [] Финальный текст политики показан юристу.

Готово! После шага 17 фича работает: бриф > этапы и часы от YandexGPT > сумма по вашей ставке и налогу. Если что-то не сходится – таблица в Приложении В.